

2520030340

SKILL-22

AIM

To run **Valgrind** to identify memory leaks and memory-related errors, fix the errors, release dynamically allocated resources, verify proper memory cleanup, and perform software testing by designing test cases, creating test scripts, automating test execution, verifying outputs, comparing actual and expected results, and generating test reports.

OBJECTIVES

1. To understand the use of Valgrind for memory debugging.
2. To identify memory leaks in a C/C++ program.
3. To detect invalid memory access and memory-related errors.
4. To properly allocate and deallocate dynamic memory.
5. To release all resources used by the program.
6. To verify that the program performs proper cleanup.
7. To design test cases for different input conditions.
8. To create test scripts for automated testing.
9. To execute multiple test cases automatically.
10. To compare expected and actual outputs.
11. To identify failed test cases.
12. To generate a test report containing the testing results.

REQUIREMENTS

- Ubuntu/Linux operating system
- GCC compiler
- Valgrind
- Terminal
- C/C++ source code
- Test input files
- Expected output files
- Test scripts

PROCEDURE

Part A: Running Valgrind

1. Open the Ubuntu terminal.
2. Create or open the required C program.
3. Compile the program using GCC.
4. Run the program normally and verify that it works correctly.
5. Execute the program using Valgrind.
6. Observe the Valgrind output displayed in the terminal.
7. Check for invalid reads and writes.
8. Check for memory leaks.
9. Identify the source of the memory errors.
10. Modify the program to fix the identified errors.
11. Ensure that dynamically allocated memory is released using `free()`.
12. Ensure that files are properly closed using `fclose()`.
13. Recompile the corrected program.
14. Run the corrected program again using Valgrind.
15. Verify that the memory errors and leaks have been removed.

Part B: Designing Test Cases

1. Identify the functionality that needs to be tested.
2. Prepare different input conditions.
3. Include normal test cases.
4. Include boundary-value test cases.
5. Include invalid-input test cases where applicable.
6. Determine the expected output for each test case.
7. Store the test inputs and expected outputs.

Part C: Creating Test Scripts

1. Create a shell script for executing the program.
2. Provide the test input to the program.
3. Capture the program's actual output.
4. Compare the actual output with the expected output.
5. Display whether the test case passed or failed.
6. Repeat the process for all test cases.

Part D: Automating Execution

1. Make the test script executable.
2. Execute the script from the terminal.
3. Run all test cases automatically.
4. Capture the output of each test case.
5. Compare the results automatically.

6. Count the number of passed and failed test cases.
7. Generate a final testing summary.

IMPORTANT VALGRIND COMMANDS

Compile the program:

```
gcc program.c -o program
```

Run normally:

```
./program
```

Run using Valgrind:

```
valgrind ./program
```

Run with detailed memory-leak information:

```
valgrind --leak-check=full ./program
```

Run with detailed error information:

```
valgrind --leak-check=full --track-origins=yes ./program
```

SAMPLE TEST SCRIPT

```
#!/bin/bash
```

```
echo "Running Test Cases..."
```

```
./program < input1.txt > actual1.txt
```

```
if diff -q actual1.txt expected1.txt > /dev/null
```

```
then
```

```
    echo "Test Case 1: PASS"
```

```
else
```

```
    echo "Test Case 1: FAIL"
```

```
fi
```

```
echo "Testing Completed."
```

Make the script executable:

```
chmod +x test.sh
```

Run the script:

```
./test.sh
```

TEST CASE TABLE

Test Case	Description	Expected Result	Status
TC01	Normal input	Correct output	PASS
TC02	Minimum input	Correct output	PASS
TC03	Maximum input	Correct output	PASS
TC04	Boundary input	Correct output	PASS
TC05	Special/edge case	Correct output	PASS

VALGRIND VERIFICATION

After correcting the program, Valgrind should indicate that there are no memory leaks or invalid memory operations.

Important information to verify:

- Invalid reads
- Invalid writes
- Use of uninitialized memory
- Memory leaks
- Allocated memory
- Freed memory
- Memory still reachable
- Error summary

RESULT

Thus, **Valgrind** was successfully used to identify and fix memory errors and memory leaks. Dynamic memory and other resources were properly released and cleanup was verified. Test cases and test scripts were designed, automated, executed, and their actual outputs were compared with the expected outputs. Finally, the test results were documented in the form of a test report.

The program was successfully executed and tested. Valgrind was used to detect memory errors and memory leaks. The output showed that all allocated memory was properly freed with 0 errors. Automated test cases were executed successfully and the expected and actual outputs were verified.

```
kruthi@LAPTOP-LP9QU3RR: ~  
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bin/jhsdb to provide /usr/bin/jhsdb (jhsdb) in auto mode  
Setting up default-jre (2:1.21-75+exp1) ...  
Setting up openjdk-21-jdk:amd64 (21.0.12.1+1-1~24.04.4) ...  
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bin/jconsole to provide /usr/bin/jconsole (jconsole) in auto mode  
Setting up default-jdk-headless (2:1.21-75+exp1) ...  
Setting up default-jdk (2:1.21-75+exp1) ...  
kruthi@LAPTOP-LP9QU3RR:~$ javac -version  
javac 21.0.12.1  
kruthi@LAPTOP-LP9QU3RR:~$ javac Skill21.java  
kruthi@LAPTOP-LP9QU3RR:~$ java Skill21  
Enter name: kruthi  
Enter age: 18  
  
User kruthi registered successfully with age 18  
End-to-end execution completed successfully.  
Program execution completed.  
kruthi@LAPTOP-LP9QU3RR:~$ java Skill21  
Enter name: kruthi  
Enter age: 150  
  
Invalid Input: Age must be between 0 and 120.  
[ERROR LOG] Age must be between 0 and 120.  
Program recovered gracefully.  
Program execution completed.  
kruthi@LAPTOP-LP9QU3RR:~$ mkdir skill22  
kruthi@LAPTOP-LP9QU3RR:~$ cd skill22  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ nano program.c  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ gcc program.c -o program  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ ./program  
Enter number of elements: 5  
Enter 5 elements:  
10 20 30 40 50  
Elements are:  
10 20 30 40 50  
kruthi@LAPTOP-LP9QU3RR:~/skill22$
```

```
kruthi@LAPTOP-LP9QU3RR: ~  
Setting up libdebuginfod-common (0.190-1.1ubuntu0.1) ...  
Setting up locales (2.39-0ubuntu8.9) ...  
Generating locales (this might take a while)...  
Generation complete.  
Setting up libsource-highlight-common (3.1.9-4.3build1) ...  
Setting up libc6-dbg:amd64 (2.39-0ubuntu8.9) ...  
Setting up libipt2 (2.0.6-1build1) ...  
Setting up libbabeltrace1:amd64 (1.5.11-3build3) ...  
Setting up libc-dev-bin (2.39-0ubuntu8.9) ...  
Setting up valgrind (1:3.22.0-0ubuntu3) ...  
Setting up libc-devtools (2.39-0ubuntu8.9) ...  
Setting up libdebuginfod1t64:amd64 (0.190-1.1ubuntu0.1) ...  
Setting up libsource-highlight4t64:amd64 (3.1.9-4.3build1) ...  
Setting up gdb (15.1-1ubuntu1~24.04.1) ...  
Setting up libc6-dev:amd64 (2.39-0ubuntu8.9) ...  
Processing triggers for man-db (2.12.0-4build2) ...  
Processing triggers for libc-bin (2.39-0ubuntu8.9) ...  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ 5  
5: command not found  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ ./program  
Enter number of elements: 5  
Enter 5 elements:  
5  
^C  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ ./program  
Enter number of elements: 5  
Enter 5 elements:  
10 20 30 40 50  
Elements are:  
10 20 30 40 50  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ ./program  
Enter number of elements: 5  
Enter 5 elements:  
10 20 30 40 50  
Elements are:  
10 20 30 40 50  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ |
```

```
kruthi@LAPTOP-LP9QU3RR: ~ × + ▾ - □ ×
Enter number of elements: 5
Enter 5 elements:
10 20 30 40 50
Elements are:
10 20 30 40 50
kruthi@LAPTOP-LP9QU3RR:~/skill22$ ./program
Enter number of elements: 5
Enter 5 elements:
10 20 30 40 50
Elements are:
10 20 30 40 50
kruthi@LAPTOP-LP9QU3RR:~/skill22$ valgrind ./program
==4552== Memcheck, a memory error detector
==4552== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==4552== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==4552== Command: ./program
==4552==
Enter number of elements: 5
Enter 5 elements:
10 20 30 40 50
Elements are:
10 20 30 40 50
==4552==
==4552== HEAP SUMMARY:
==4552==      in use at exit: 0 bytes in 0 blocks
==4552==    total heap usage: 3 allocs, 3 frees, 2,068 bytes allocated
==4552==
==4552== All heap blocks were freed -- no leaks are possible
==4552==
==4552== For lists of detected and suppressed errors, rerun with : -s
==4552== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
kruthi@LAPTOP-LP9QU3RR:~/skill22$ |
```

```
kruthi@LAPTOP-LP9QU3RR: ~ × + ▾ - □ ×
==4552==      in use at exit: 0 bytes in 0 blocks
==4552==    total heap usage: 3 allocs, 3 frees, 2,068 bytes allo
cated
==4552==
==4552== All heap blocks were freed -- no leaks are possible
==4552==
==4552== For lists of detected and suppressed errors, rerun with
: -s
==4552== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0
from 0)
kruthi@LAPTOP-LP9QU3RR: ~/skill22$ valgrind --leak-check=full --t
rack-origins=yes ./program
==4555== Memcheck, a memory error detector
==4555== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Sewar
d et al.
==4555== Using Valgrind-3.22.0 and LibVEX; rerun with -h for cop
yright info
==4555== Command: ./program
==4555==
Enter number of elements: 5
Enter 5 elements:
10 20 30 40 50
Elements are:
10 20 30 40 50
==4555==
==4555== HEAP SUMMARY:
==4555==      in use at exit: 0 bytes in 0 blocks
==4555==    total heap usage: 3 allocs, 3 frees, 2,068 bytes allo
cated
==4555==
==4555== All heap blocks were freed -- no leaks are possible
==4555==
==4555== For lists of detected and suppressed errors, rerun with
: -s
==4555== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0
from 0)
kruthi@LAPTOP-LP9QU3RR: ~/skill22$ |
```

```
kruthi@LAPTOP-LP9QU3RR: ~  
yright info  
==4555== Command: ./program  
==4555==  
Enter number of elements: 5  
Enter 5 elements:  
10 20 30 40 50  
Elements are:  
10 20 30 40 50  
==4555==  
==4555== HEAP SUMMARY:  
==4555==      in use at exit: 0 bytes in 0 blocks  
==4555==    total heap usage: 3 allocs, 3 frees, 2,068 bytes allocated  
==4555==  
==4555== All heap blocks were freed -- no leaks are possible  
==4555==  
==4555== For lists of detected and suppressed errors, rerun with  
: -s  
==4555== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0  
from 0)  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ nano input.txt  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ nano test.sh  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ chmod +x test.sh  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ ./test.sh  
=====
```

SKILL-22 TEST

```
=====
```

Program Output:
Enter number of elements: Enter 5 elements:
Elements are:
10 20 30 40 50

Test Case 1: PASS

```
=====
```

Testing Completed

```
=====
```

kruthi@LAPTOP-LP9QU3RR:~/skill22\$ |

```
kruthi@LAPTOP-LP9QU3RR: ~ × + ▢ ×
10 20 30 40 50
Elements are:
10 20 30 40 50
==4555==
==4555== HEAP SUMMARY:
==4555==       in use at exit: 0 bytes in 0 blocks
==4555==    total heap usage: 3 allocs, 3 frees, 2,068 bytes allo
cated
==4555==
==4555== All heap blocks were freed -- no leaks are possible
==4555==
==4555== For lists of detected and suppressed errors, rerun with
: -s
==4555== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0
from 0)
kruthi@LAPTOP-LP9QU3RR:~/skill22$ nano input.txt
kruthi@LAPTOP-LP9QU3RR:~/skill22$ nano test.sh
kruthi@LAPTOP-LP9QU3RR:~/skill22$ chmod +x test.sh
kruthi@LAPTOP-LP9QU3RR:~/skill22$ ./test.sh
=====
                SKILL-22 TEST
=====
Program Output:
Enter number of elements: Enter 5 elements:
Elements are:
10 20 30 40 50

Test Case 1: PASS
=====
Testing Completed
=====
kruthi@LAPTOP-LP9QU3RR:~/skill22$ nano input2.txt
kruthi@LAPTOP-LP9QU3RR:~/skill22$ ./program < input2.txt
Enter number of elements: Enter 3 elements:
Elements are:
5 15 25
kruthi@LAPTOP-LP9QU3RR:~/skill22$ |
```

```

kruthi@LAPTOP-LP9QU3RR: ~
Elements are:
10 20 30 40 50
==4555==
==4555== HEAP SUMMARY:
==4555==    in use at exit: 0 bytes in 0 blocks
==4555==  total heap usage: 3 allocs, 3 frees, 2,068 bytes allocated
==4555==
==4555== All heap blocks were freed -- no leaks are possible
==4555==
==4555== For lists of detected and suppressed errors, rerun with: -s
==4555== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
kruthi@LAPTOP-LP9QU3RR:~/skill22$ nano input.txt
kruthi@LAPTOP-LP9QU3RR:~/skill22$ nano test.sh
kruthi@LAPTOP-LP9QU3RR:~/skill22$ chmod +x test.sh
kruthi@LAPTOP-LP9QU3RR:~/skill22$ ./test.sh
=====
                SKILL-22 TEST
=====
Program Output:
Enter number of elements: Enter 5 elements:
Elements are:
10 20 30 40 50

Test Case 1: PASS
=====
Testing Completed
=====
kruthi@LAPTOP-LP9QU3RR:~/skill22$ nano input2.txt
kruthi@LAPTOP-LP9QU3RR:~/skill22$ ./program < input2.txt
Enter number of elements: Enter 3 elements:
Elements are:
5 15 25
kruthi@LAPTOP-LP9QU3RR:~/skill22$ nano input3.txt
kruthi@LAPTOP-LP9QU3RR:~/skill22$ |

```

```
kruthi@LAPTOP-LP9QU3RR: ~  
10 20 30 40 50  
  
Test Case 1: PASS  
=====
```

Testing Completed

```
=====
```

kruthi@LAPTOP-LP9QU3RR:~/skill22\$ nano input2.txt

kruthi@LAPTOP-LP9QU3RR:~/skill22\$./program < input2.txt

Enter number of elements: Enter 3 elements:

Elements are:

5 15 25

kruthi@LAPTOP-LP9QU3RR:~/skill22\$ nano input3.txt

kruthi@LAPTOP-LP9QU3RR:~/skill22\$ valgrind --leak-check=full ./program < input2.txt

```
==4591== Memcheck, a memory error detector  
==4591== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.  
==4591== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info  
==4591== Command: ./program  
==4591==  
Enter number of elements: Enter 3 elements:  
Elements are:  
5 15 25  
==4591==  
==4591== HEAP SUMMARY:  
==4591==    in use at exit: 0 bytes in 0 blocks  
==4591==   total heap usage: 3 allocs, 3 frees, 5,132 bytes allocated  
==4591==  
==4591== All heap blocks were freed -- no leaks are possible  
==4591==  
==4591== For lists of detected and suppressed errors, rerun with  
: -s  
==4591== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)  
kruthi@LAPTOP-LP9QU3RR:~/skill22$ |
```